Lab 04 — Deploy a Web Application on Render (PaaS)

DevOps & Cloud Computing · 5th-year Software Engineering · Nov 11, 2026 · @mehdi

Objective and results

A Node.js/Express web app was built locally, pushed to GitHub and deployed on Render, where it is live and redeploys automatically on every push to `main`.

Student: Cheikhaoui Ahmed Mahdi

Tools: Render (PaaS), Node.js 24, npm 11, Express 5, Git, GitHub

Deliverable	Value
GitHub repository	github.com/cheihkaouimahdi-hub/Render
Live Render URL	render-262g.onrender.com
Render service	Web Service "Render" · Node runtime · Free instance · branch `main`

Part 1 — Create the Node.js application

The app ran locally with `npm start` and returned "Hello from Render!" at http://localhost:3000.

1. `npm init -y` created `package.json`.
2. `npm install express` installed Express 5.3.0 (65 packages, 0 vulnerabilities).
3. `index.js` was created with the code below.
4. The `scripts` section of `package.json` was set to `"start": "node index.js"`.
5. `npm start` started the server on port 3000.
6. http://localhost:3000 displayed "Hello from Render!".

`index.js`:

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.send("Hello from Render!");
});

app.listen(PORT, "0.0.0.0", () => {
  console.log(`Server listening on port ${PORT}`);
});
```

`package.json` (relevant part):

```
{
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^5.3.0"
  }
}
```

Terminal output of the local run:

```
> tp4@1.0.0 start
> node index.js

Server listening on port 3000
```

Figure 1 — Local run: *screenshot to add (terminal running `npm start` + browser at http://localhost:3000).*

Part 2 — Publish the code on GitHub

The project was pushed to the `main` branch of [cheihkaouimahdi-hub/Render](). A `.gitignore` keeps `node_modules/` and `.env` files out of the repository.

`.gitignore`:

```
node_modules/
.env
.env.*
npm-debug.log*
```

Git workflow:

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/cheihkaouimahdi-hub/Render.git
git push -u origin main
```

Files in the repository: `.gitignore`, `README.md`, `index.js`, `package.json`, `package-lock.json`. No `node_modules/`, passwords, API keys, tokens or `.env` files were published.

Commit	Date (GMT+1)	Message
`d75ef73`	2026-11-11 19:22	Update welcome message
`1368393`	2026-11-11 19:10	initialize package.json with express dependency and start script
`4f41abf`	2026-11-11 19:09	first commit

The repository is named `Render` instead of the suggested `node-render-demo`; this does not affect the deployment.

Part 3 — Deploy on Render

The first deploy (commit `1368393`) succeeded in 28.9 s on 2026-11-11 at 19:18 GMT+1, and the app went live at https://render-262g.onrender.com.

Setting	Value
Service type	Web Service
Repository	cheihkaouimahdi-hub/Render, branch `main`
Runtime	Node (Node.js 24.22.1, Render default)
Build Command	`npm install`
Start Command	`npm start`
Instance type	Free
Environment variables	none added; Render supplies `PORT`

Key lines from the deploy log:

```
==> Cloning from https://github.com/cheihkaouimahdi-hub/Render
==> Using Node.js version 24.22.1 (default)
==> Running build command 'npm install'...
added 65 packages, and audited 66 packages in 693ms
==> Build successful
==> Running 'npm start'
Server listening on port 10000
==> Your service is live
==> Available at your primary URL https://render-262g.onrender.com
```

The server listened on port 10000, not 3000: Render injected its own port through `process.env.PORT`.

Figure 2 — Render deploy log: build successful, service live.

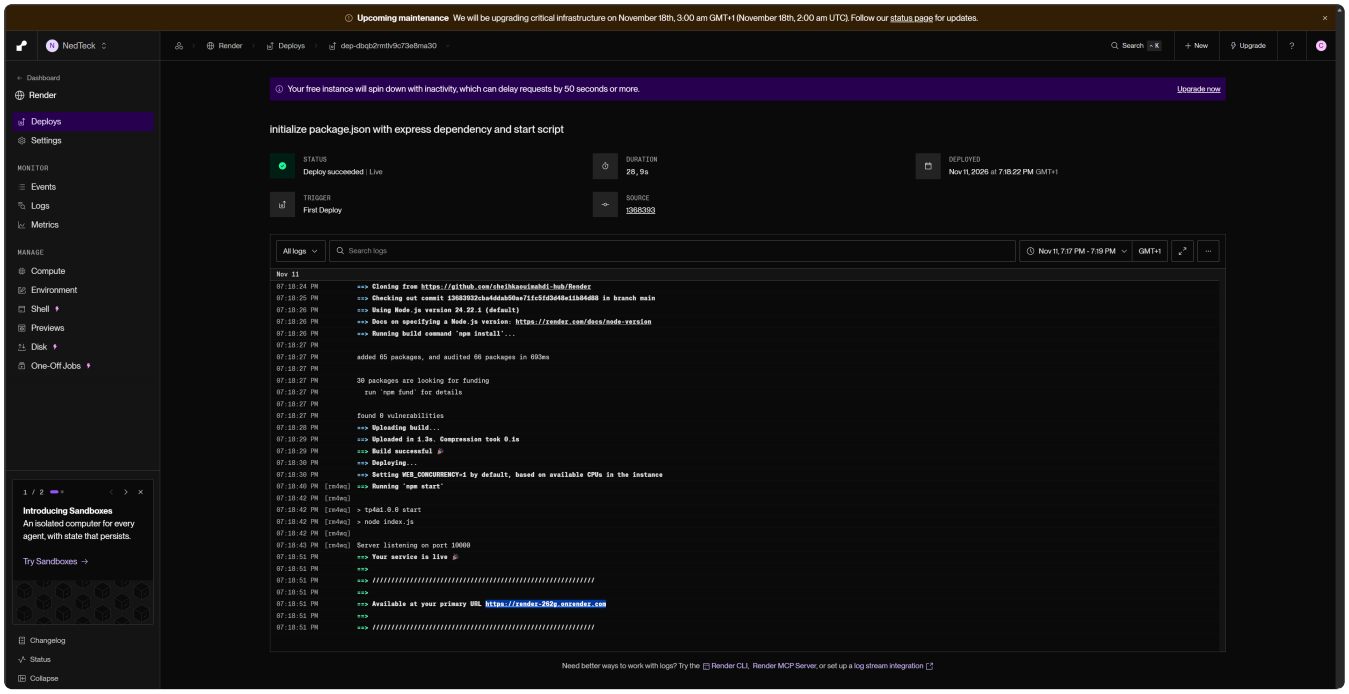
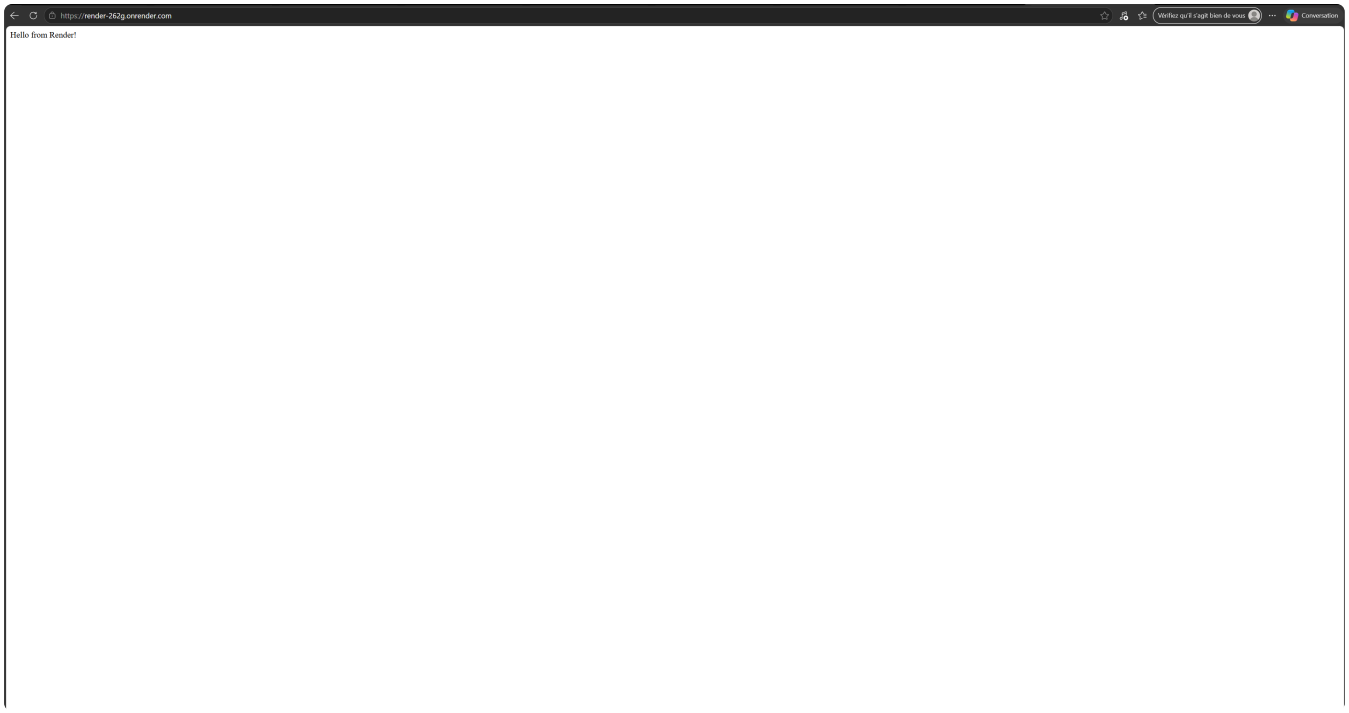


Figure 3 — Live app at https://render-262g.onrender.com showing "Hello from Render!".



Part 4 — Verify and update the deployment

After one `git push`, Render redeployed automatically in about one minute and the live page showed the new message.

1. The live URL was opened and showed "Hello from Render!" (Figure 3).
2. The message on line 6 of `index.js` was changed:

```
res.send("Hello from Render!/i am cheikhaoui ahmed mahdi");
```

3. The change was committed and pushed:

```
git add index.js
git commit -m "Update welcome message"
git push
```

4. Render detected the new commit and redeployed it without any manual action:

Time (GMT+1)	Event
2026-11-11 19:23	Deploy live for `d75ef73`: Update welcome message
2026-11-11 19:22	Deploy started for `d75ef73`: "New commit via Auto-Deploy"
2026-11-11 19:18	Deploy live for `1368393` (first deploy)

Time (GMT+1)	Event
2026-11-11 19:18	First deploy started for `1368393`

Figure 4 — Render Events: the push triggered an Auto-Deploy of commit `d75ef73`.

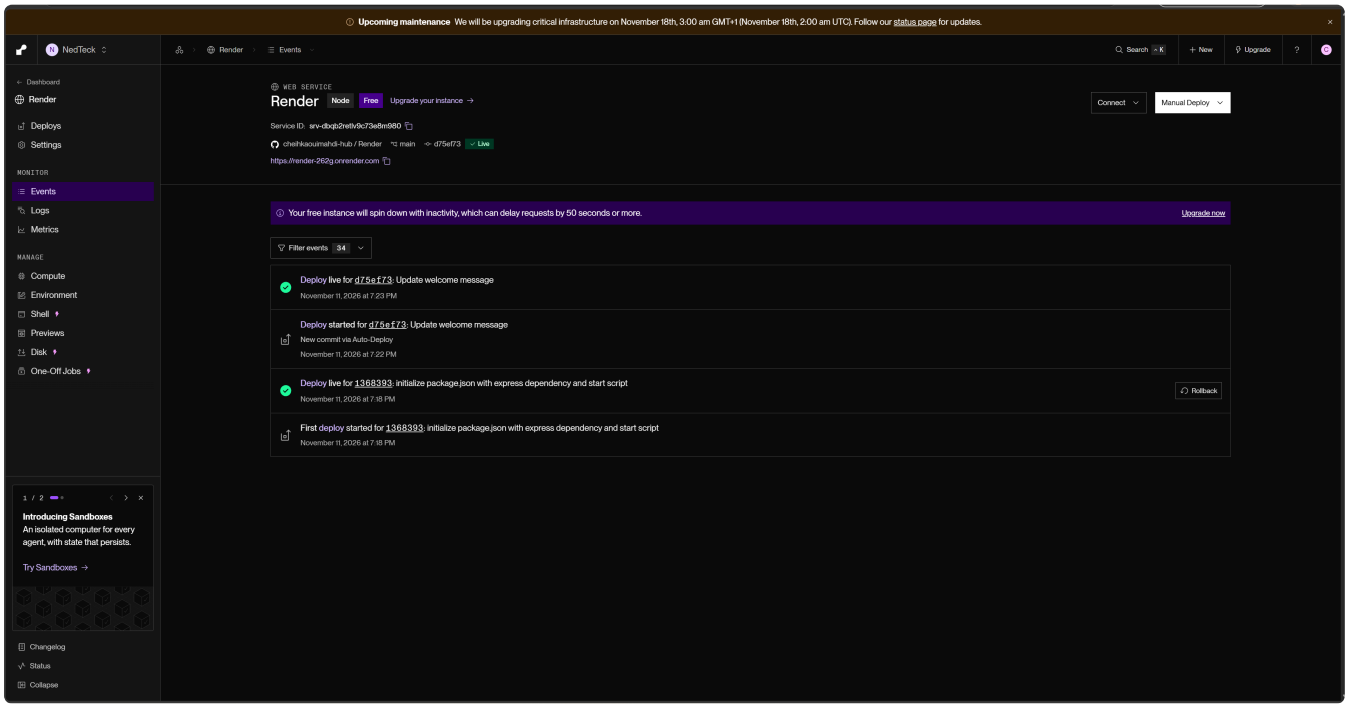


Figure 5 — Live app after redeployment showing "Hello from Render!/i am cheikhaoui ahmed mahdi".



Questions

1. What tasks does Render manage that you would otherwise manage on a virtual machine?

Render handles all of the infrastructure; only the application code and two commands came from us.

Task	On a virtual machine (IaaS)	On Render (PaaS)
Server provisioning, hardware	Create and size the VM	Managed
Operating system, security patches	Install and update it yourself	Managed
Node.js runtime	Install and upgrade Node.js	Provided (Node.js 24.22.1)
Build and deploy	Copy code, run `npm install`, restart by hand	Clones from GitHub, builds, deploys
Process management	Set up systemd/PM2, restart on crash	Starts `npm start`, restarts on failure
Networking	Open firewall ports, reverse proxy (Nginx)	Public URL and routing to `PORT`
HTTPS and domain	Buy/configure DNS, install TLS certificates	`*.onrender.com` with HTTPS included
Logs, monitoring, rollback	Install your own tools	Logs, Metrics, Events, Rollback built in
Scaling	Resize or add VMs, add a load balancer	Change instance type or count

2. What are the roles of Node.js and Express in this application?

- **Node.js** is the JavaScript runtime that executes `index.js` on the server. It provides the HTTP networking and the event loop that handles requests; npm, which comes with it, installs the dependencies.
- **Express** is a web framework built on top of Node.js. It creates the HTTP server (`app.listen`), defines the route for `/` (`app.get`) and sends the response (`res.send`).

3. Why does the code use `process.env.PORT` instead of relying only on port 3000?

Render chooses the port at runtime and passes it in the `PORT` environment variable; during this deploy it was 10000. Render's proxy forwards public traffic only to that port, so an app hard-coded to 3000 would not be reachable and the deploy would fail. The `|| 3000` fallback keeps the app working locally, where `PORT` is not set. Binding to `"0.0.0.0"` lets the server accept connections from outside the container, not only from localhost.

4. What role does GitHub play in this deployment workflow?

GitHub is the single source of the code that Render deploys:

- **Source repository:** Render clones `cheihkaouimahdi-hub/Render` (branch `main`) at every build.
- **Deploy trigger:** each push notifies Render, which starts an Auto-Deploy, as seen for commit `d75ef73`.
- **Version history:** every deploy is linked to a commit, so a previous version can be restored with Render's Rollback button.
- **Backup and collaboration:** the code is stored remotely and can be shared with other developers.

5. Deliverables and why this is PaaS

Deliverable	Value / evidence
GitHub repository URL	https://github.com/cheihkaouimahdi-hub/Render
Live Render URL	https://render-262g.onrender.com
Local deployment evidence	Part 1 terminal output and Figure 1
Cloud deployment evidence	Figures 2–5 (deploy log, live page, Auto-Deploy events, updated page)

This is PaaS because we supplied only the code and two commands (`npm install`, `npm start`). Render provided and ran everything underneath: the server, operating system, Node.js runtime, build pipeline, networking, HTTPS, public URL and redeploys. We never created, logged into or configured a machine, which is the difference from IaaS, where we would manage the VM ourselves.

Conclusion

The Express app moved from `localhost:3000` to a public HTTPS URL on Render in under 30 seconds of build time. A single `git push` then delivered a new version without touching any server, which shows the main benefit of PaaS: developers focus on code while the platform handles infrastructure and deployment.